



Dnyaan Prasad Global University
Dr. D. Y. Patil Unitech Society
School of Technology and Research

Academic year 2025-26 (semester I)

DIV:-C

- *Mrunalini Patil* *ROLL.NO-25FC318*
- *Sandhya Tembhrune* *ROLL.NO-25FC310*

TITLE: Menu-Driven Calculator Program In C

AIM

To design and implement a **menu-driven scientific calculator** in the C programming language that allows the user to perform a variety of mathematical operations including arithmetic, modulus, square root, power, factorial, and trigonometric calculations using standard library functions.

OBJECTIVES

1. To understand menu-driven programming using loops and decision-making structures.
2. To make use of the `<math.h>` library for performing advanced mathematical functions.
3. To handle different data types, including integers and floating-point numbers.
4. To implement error-checking for invalid inputs such as division by zero or negative square roots.
5. To create a user-friendly, interactive calculator that repeatedly displays a menu until the user chooses to exit.
6. To strengthen understanding of iterative structures (do-while loop) and the switch-case decision-making mechanism in C.

THEORY

1. Menu-Driven Programming

Menu-driven programming is a technique where the program repeatedly displays a list of choices to the user and performs the selected operation. It enhances user interactivity and makes the program flexible and structured. In this program, a menu is displayed inside a do-while loop so that the calculator continues to operate until the user selects the “Exit” option.

2. Decision-Making Using Switch-Case

The switch-case structure is ideal for handling multiple user choices. Each menu option corresponds to a case label. This makes the program more readable, organized, and efficient than using multiple if-else statements.

For example, different cases handle operations such as:

- Addition
- Subtraction
- Multiplication
- Division
- Modulus
- Square Root
- Power
- Factorial
- Trigonometric functions

If the user enters an invalid option, the default case handles it gracefully.

3. Use of Standard Library Functions

The program uses mathematical functions from `<math.h>`:

Function Description

`sqrt(x)` Computes square root

Function Description

`pow(x,y)` Computes x raised to power y

`sin(x)` Computes sine of angle in radians

`cos(x)` Computes cosine

`tan(x)` Computes tangent

Trigonometric functions in C require **radians**, so the program converts degrees to radians using:

$\text{radians} = \text{degrees} \times (\pi / 180)$

`#define PI 3.14159265` is used to represent the value of π .

4. Arithmetic and Modulus Operations

Basic arithmetic operations—addition, subtraction, multiplication, and division—are performed using double-precision variables since users may enter decimal numbers.

However, modulus (%) works only on integers, so separate integer variables are used. The program also checks for **division by zero**, which is mathematically undefined.

5. Square Root and Power

The square root function (`sqrt()`) is only valid for non-negative numbers, so the program checks whether the input is negative.

Similarly, power calculation is performed with the `pow()` function, supporting both integer and floating-point exponents.

6. Factorial Calculation

Factorial is the product of all positive integers up to a given number:

$$n! = 1 \times 2 \times 3 \times \dots \times n$$

Since factorial grows rapidly, a long long type is used. Negative factorial is not allowed, so the program validates the input.

7. Trigonometric Functions

The calculator also supports trigonometric operations:

- Sine
- Cosine
- Tangent

Tangent values at angles like 90° , 270° , etc., are undefined mathematically, so the program accounts for this. This improves reliability and prevents incorrect output

8. Repetition and Exit Mechanism

The entire program is placed inside an infinite do-while loop. The loop continues running until the user selects option **10**, at which point:

```
exit(0);
```

is executed to terminate the program immediately.

This structure makes the calculator easy to use and continuously available until the user chooses to stop.

ALGORITHM

STEP 1: Start the program.

STEP 2: Display the main menu showing all calculator options:

- Addition
- Subtraction
- Multiplication
- Division
- Modulus
- Square Root
- Power
- Factorial
- Trigonometric Functions
- Exit

STEP 3: Ask the user to enter their choice (1–10).

STEP 4: If the user selects **Option 10 (Exit)**, display an exit message and terminate the program.

STEP 5: Use a **switch-case** structure to handle the selected operation.

STEP 6: If the user chooses **Addition, Subtraction, Multiplication, or Division (Choices 1–4)**:

- Ask for two numbers (floating-point allowed).
- Perform the selected arithmetic operation.
- For division, check if the second number is zero; if zero, display an error.

STEP 7: If the user chooses **Modulus (Choice 5)**:

- Ask for two integers.
- Check if the second integer is zero; if zero, show an error.
- Otherwise, compute $\text{num1} \% \text{num2}$.

STEP 8: If the user chooses **Square Root (Choice 6):**

- Ask for a number.
- If the number is negative, display an error message.
- Otherwise, compute the square root using `sqrt()`.

STEP 9: If the user chooses **Power (Choice 7):**

- Ask for the base and exponent.
- Compute the result using the `pow()` function.

STEP 10: If the user chooses **Factorial (Choice 8):**

- Ask for a non-negative integer.
- If the number is negative, display an error.
- Otherwise, compute factorial by multiplying numbers from 1 to n .

STEP 11: If the user chooses **Trigonometric Functions (Choice 9):**

- Display the trigonometry sub-menu (Sine, Cosine, Tangent).
- Ask for the user's sub-choice.
- Ask for an angle in degrees.
- Convert degrees to radians using:
- $\text{radians} = \text{degrees} \times (\text{PI} / 180)$
- Compute the selected trigonometric function.
- For tangent, check if the angle results in an undefined value (like 90° , 270°).

STEP 12: Display the final computed result for the selected operation.

STEP 13: Return to the main menu and allow the user to choose another operation.

STEP 14: Repeat STEPS 2 through 13 until the user chooses to exit.

STEP 15: Stop the program.

CODE:-

```
#include <stdio.h>

#include <math.h>

#include <stdlib.h>

#define PI 3.14159265

int main() {

    int choice, num1_int, num2_int;

    double num1_double, num2_double, result;

    do {

        printf("\n===== Menu-Driven Calculator =====\n");

        printf("1. Addition\n");

        printf("2. Subtraction\n");

        printf("3. Multiplication\n");

        printf("4. Division\n");

        printf("5. Modulus (for integers)\n");

        printf("6. Square Root\n");

        printf("7. Power (Exponentiation)\n");

        printf("8. Factorial\n");

        printf("9. Trigonometric Functions (Sine, Cosine, Tangent)\n");

        printf("10. Exit\n");

        printf("Enter your choice (1-10): ");

        scanf("%d", &choice);

        // Exit condition
```

```
if (choice == 10) {  
    printf("Exiting Calculator.\n");  
    exit(0);  
}
```

```
switch (choice) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
        printf("Enter two numbers: ");  
        scanf("%lf %lf", &num1_double, &num2_double);  
        if (choice == 1) result = num1_double + num2_double;  
        else if (choice == 2) result = num1_double - num2_double;  
        else if (choice == 3) result = num1_double * num2_double;  
        else {  
            if (num2_double != 0)  
  
            {  
                result = num1_double / num2_double;  
            } else {  
                printf("Error: Division by zero is not allowed.\n");  
                continue;  
            }  
        }  
        printf("Result = %.4lf\n", result);  
        break;
```

case 5:

```
printf("Enter two integers (num1 %% num2): ");  
scanf("%d %d", &num1_int, &num2_int);  
if (num2_int != 0) {  
    result = num1_int % num2_int;  
    printf("Result = %d\n", (int)result);  
} else {  
    printf("Error: Division by zero is not allowed.\n");  
}  
break;
```

case 6:

```
printf("Enter a number to find the square root: ");  
scanf("%lf", &num1_double);  
if (num1_double >= 0) {  
    result = sqrt(num1_double);  
    printf("Result = %.4lf\n", result);  
} else {  
    printf("Error: Square root of a negative number is not possible.\n");  
}  
break;
```

case 7:

```
printf("Enter base and exponent (base^exponent): ");  
scanf("%lf %lf", &num1_double, &num2_double);  
result = pow(num1_double, num2_double);  
printf("Result = %.4lf\n", result);  
break;
```

case 8:

```
printf("Enter a non-negative integer for factorial: ");  
scanf("%d", &num1_int);  
if (num1_int >= 0) {  
    long long factorial = 1;  
    for (int i = 1; i <= num1_int; ++i) {  
        factorial *= i;  
    }  
    printf("Result = %lld\n", factorial);  
} else {  
    printf("Error: Factorial is only for non-negative integers.\n");  
}  
break;
```

case 9: {

```
int trig_choice;  
  
double angle_degrees, angle_radians;  
  
printf("\n Trigonometry Menu:\n");  
printf(" 1. Sine\n 2. Cosine\n 3. Tangent\n");  
printf(" Enter your trigonometry choice (1-3): ");  
scanf("%d", &trig_choice);  
  
printf(" Enter angle in degrees: ");  
scanf("%lf", &angle_degrees);  
  
  
// Convert degrees to radians for math.h functions  
angle_radians = angle_degrees * (PI / 180.0);
```

```

if (trig_choice == 1) {
    result = sin(angle_radians);
    printf(" sin(%.2lf degrees) = %.4lf\n", angle_degrees, result);
} else if (trig_choice == 2) {
    result = cos(angle_radians);
    printf(" cos(%.2lf degrees) = %.4lf\n", angle_degrees, result);
}
else if (trig_choice == 3)
{
    // Check for tangent of 90, 270 degrees etc.
    if ((int)angle_degrees % 180 == 90 || (int)angle_degrees % 180 == -90) {
        printf(" tan(%.2lf degrees) is undefined.\n", angle_degrees);
    }
    else
    {
        result = tan(angle_radians);
        printf(" tan(%.2lf degrees) = %.4lf\n", angle_degrees, result);
    }
}
else
{
    printf(" Invalid trigonometry choice.\n");
}
break;
}

default:
    printf("Invalid choice, please try again.\n");

```

```
    }  
    } while (1);  
    return 0;  
}
```

SAMPLE INPUT / OUTPUT

Example 1: Addition

Input:

Enter your choice: 1

Enter two numbers: 15 30

Output:

Result = 45.0000

Example 2: Division by Zero

Input:

Enter your choice: 4

Enter two numbers: 10 0

Output:

Error: Division by zero is not allowed.

Example 3: Factorial

Input:

Enter your choice: 8

Enter a non-negative integer for factorial: 5

Output:

Result = 120

Example 4: Trigonometry

Input:

Enter your choice: 9

Enter trigonometry choice: 1

Enter angle in degrees: 30

Output:

$\sin(30.00 \text{ degrees}) = 0.5000$

REAL-LIFE APPLICATIONS

1. Scientific and engineering calculations.
2. Educational tools for teaching basic mathematics.
3. Basis for developing GUI-based or mobile calculator applications.
4. Embedded systems that require mathematical processing.
5. Useful for students learning C programming and numerical methods.
6. Can be extended to perform statistical or matrix operations.

ADVANTAGES

1. User-friendly and interactive due to the menu-driven structure.
2. Supports multiple types of calculations within a single program.
3. Incorporates advanced mathematical operations through `<math.h>`.
4. Proper error-handling ensures safe program execution.
5. Easily expandable—more operations can be added using new cases.
6. Portable and efficient, works on any system that supports C.

CONCLUSION

The menu-driven calculator program demonstrates effective use of loops, decision structures, mathematical libraries, and user interaction in the C language. By integrating both basic and advanced mathematical operations, the program serves as a

comprehensive calculator capable of performing arithmetic, logical, and scientific computations.

The use of error-checking mechanisms ensures that invalid inputs (such as division by zero, negative values for square roots, or undefined trigonometric angles) are handled gracefully. This enhances reliability and robustness. Through this assignment, foundational programming concepts such as control flow, data types, iterative structures, and modular design are reinforced. Overall, the program is an excellent example of interactive, structured, and efficient programming in C.